B DATA 200 Notes

Introduction to Data Studies, Autumn 2026

Derek Nguyen

October 29, 2026

Table of contents

1 Workflow & Reminders **3**
- 1.1 Packages . . . 3
- 1.2 Project Setup . . . 3
- 1.3 Keyboard Shortcuts . . . 3
- 1.4 Chunk Options . . . 4
- 1.5 Assignment Operators: `<-` versus `=` . . . 4
- 1.6 Reading Data . . . 5

2 Data Basics **5**
- 2.1 Quick Reference: Inspecting / Overview . . . 5
- 2.2 Case-by-Variable Format . . . 6
- 2.3 Column Types . . . 6
- 2.4 The dplyr Rules . . . 6
- 2.5 The Pipe `|>` . . . 7

3 Rows & Columns **7**
- 3.1 Quick Reference: Row Operations . . . 7
- 3.2 Quick Reference: Column Operations . . . 8
- 3.3 Worked Example: Point Values for a Deck of Cards (`mutate()`) . . . 8
- 3.4 Worked Example: Flights to Seattle with the Longest Delays . . . 9

4 Groups & Summaries **10**
- 4.1 Quick Reference . . . 10
- 4.2 Counting: `count()` versus `tally()` versus `n()` . . . 11
- 4.3 Worked Example: Most Delayed Departures by NYC Airport . . . 12

5 Tidying & Reshaping **13**
- 5.1 Quick Reference . . . 13
- 5.2 Wide → Long: `pivot_longer()` . . . 13
  - 5.2.1 Splitting column names into several variables . . . 14
- 5.3 Long → Wide: `pivot_wider()` . . . 14
- 5.4 Cleaning Messy Survey Answers . . . 15

6 Joins & Relational Data **15**
- 6.1 Quick Reference . . . 16

6.2 Example Tables . . . 17
6.3 Keys . . . 17
6.3.1 Relationships . . . 18
6.4 Mutating Joins (add columns) . . . 18
6.5 Filtering Joins (keep or drop rows, no new columns) . . . 19
6.6 Set Operations (compare whole rows) . . . 20

7 Plots with ggplot2 **21**
7.1 Quick Reference: Geoms . . . 21
7.1.1 Examples . . . 22
7.2 Trend Lines (`geom_smooth()`) . . . 22
7.3 Overplotting: Jitter, Transparency, Count . . . 23
7.4 Reference Lines and Axis Limits . . . 24
7.5 Quick Reference: Plot Styling . . . 25

8 Exploratory Data Analysis **26**
8.1 Variation and Typical Values . . . 27
8.1.1 Measures of center and spread . . . 28
8.2 Unusual Values . . . 28
8.3 Covariation . . . 29
8.4 Patterns and Models . . . 30

9 Choosing Plots & Models **30**
9.1 Choosing a Plot . . . 30
9.2 Correlation . . . 31
9.3 Simple Linear Regression . . . 31
9.3.1 Worked Example: Study Hours and Exam Scores . . . 31
9.4 Categorical Predictor = Comparing Group Means . . . 32
9.4.1 Where ANOVA fits . . . 33

10 Study Design & Data Ethics **34**
10.1 Study Designs . . . 34
10.2 Ways to Collect Data . . . 34
10.3 Consent & Privacy . . . 35
10.4 Writing Unbiased Survey Questions . . . 35

11 Common Pitfalls & Troubleshooting **35**
11.1 `=` instead of `==` . . . 35
11.2 Forgetting `na.rm = TRUE` . . . 36
11.3 Using the `&` and `|` Operators . . . 36
11.4 Forgetting to Save with `<-` . . . 37
11.5 Filtering Before a Step That Needs the Rows . . . 38
11.6 Quoting Column Names in Pivots . . . 38
11.7 Joins That Add Rows . . . 38
11.8 `NA` Is Not Zero . . . 39

12 Reflection & To-do **39**
12.1 Notes #2 Reflection . . . 39
12.2 Notes #1 Reflection (archived) . . . 40
12.3 To-do List . . . 41
12.3.1 Notes #2 - October 29, 2026 . . . 41

12.3.2 Done since Notes #1 41

13 Feedback Log **41**
13.1 Notes #1 - October 20, 2026 41

Online version, with search: https://dereknguyenn.github.io/cs-notes/bdata200/

1 Workflow & Reminders

1.1 Packages

- `install.packages("package_name")` installs a package **once per computer**.
  - Typed in the Console, never in a `.qmd` (it would reinstall on every render)
  - `package_name` is wrapped in quotes
- `library(package_name)` loads a package and goes in **every document**
  - !! On this site, every page is its own R session, so each page loads its own packages

1.2 Project Setup

From the Day 4 activity, the workflow for every new `.qmd`:

1. Open the `.Rproj` first, so file paths like `read_csv("titanic.csv")` work without a full path
2. First chunk: load packages
3. Second chunk: import the data and give it a name
4. Meet the data (`glimpse()`, `head()`, check for `NA`s) before wrangling or plotting

Keep one purpose per chunk. Loading packages and importing data are separate chunks.

1.3 Keyboard Shortcuts

I want to…	Shortcut
Run the current line or selection	Cmd + Enter
Run the whole current chunk	Cmd + Shift + Enter
Insert a code chunk	Cmd + Option + I
Type the pipe `|>`	Cmd + Shift + M
Type the assignment arrow `<-`	Option + -
Render the document	Cmd + Shift + K
See all keyboard shortcuts (overlay)	Option + Shift + K

1.4 Chunk Options

Chunk options go on the first lines inside a code chunk and start with `#|`.

> In RStudio, chunk options can be easily toggled with the cogwheel icon under each code chunk.

Option	What it does	When I use it
`#| eval: false`	Shows the code but never runs it	`View()`, `install.packages()`, code I only want to display
`#| echo: false`	Runs the code and shows the output, but hides the code	A finished plot in a report where the reader doesn't need code
`#| include: false`	Runs the code but hides both code and output	Setup chunks (loading packages, importing data)
`#| warning: false`	Hides warnings (the code still runs)	Known, harmless warnings, e.g. `geom_smooth()` messages
`#| message: false`	Hides messages, e.g. the startup text from `library(tidyverse)`	Package loading and `read_csv()` column-type messages
`#| error: true`	Keeps rendering even if the chunk errors, and shows the error	Demonstrating a mistake on purpose (see Pitfalls)
`#| label: name`	Names the chunk	Finding a chunk in the outline; cross-referencing figures

```
# A tibble: 3 x 2
  drv       n
  <chr> <int>
1 4       103
2 f       106
3 r        25
```

!! Warnings vs. messages vs. errors:

- **Message**: just information (e.g. which columns `read_csv()` guessed)
- **Warning**: something worth checking, but the code still ran (e.g. rows removed because of `NA`s)
- **Error**: the code stopped and there is no result

Hide messages freely. Only hide a warning after reading it once and deciding it's harmless.

1.5 Assignment Operators: `<-` versus `=`

Both store a value: `x <- 5` and `x = 5` do the same thing.

Convention: use `<-` for assignment.

- It keeps "store this" (`<-`) visually separate from "is this equal?" (`==`)
- `=` is still used to **name arguments** inside a function: `mean(x, na.rm = TRUE)`
- `=` also works for assignment and some people (including my instructor) use it because it's quicker. Pick one and be consistent.

```r
# assignment: store the result
jan_flights <- flights |> filter(month == 1)

# = names an argument; == checks equality
jan_flights |> summarize(avg_delay = mean(dep_delay, na.rm = TRUE))
```

```
# A tibble: 1 x 1
  avg_delay
      <dbl>
1      10.0
```

1.6 Reading Data

- `read_csv("titanic.csv")` (with an underscore) is the tidyverse version and returns a tibble
- `read.csv()` (with a dot) is base R and returns a plain data frame, so **stick with `read_csv()`**
- `read_csv()` also reads straight from a URL, e.g. TidyTuesday data on GitHub

2 Data Basics

2.1 Quick Reference: Inspecting / Overview

Use Case	Function	Example
See the first n rows	`head()`	`head(mpg, n = 10)`
See the last n rows	`tail()`	`tail(mpg, n = 3)`
See every column, its type and first values	`glimpse()`	`glimpse(flights)`
Summary stats for each column	`summary()`	`mpg |> select(hwy) |> summary()`
See the structure of the data	`str()`	`str(mpg)`
List the column names	`names()`	`names(mpg)`
Count the rows	`nrow()`	`nrow(flights)`
Count missing values in every column	`across()` inside `summarize()`	see the chunk below
Open a spreadsheet view	`View()`	`View(flights)` !! keep out of the render with `eval: false`
Read the help page for a dataset/function	`?`	`?mpg` (in the Console)

```r
# Count missing values in every column
flights |>
  summarize(across(everything(), ~ sum(is.na(.))))
```

```
# A tibble: 1 x 19
   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
  <int> <int> <int>    <int>          <int>     <int>    <int>          <int>
```

```
1     0     0     0     8255              0      8255     8713              0
# i 11 more variables: arr_delay <int>, carrier <int>, flight <int>,
#   tailnum <int>, origin <int>, dest <int>, air_time <int>, distance <int>,
#   hour <int>, minute <int>, time_hour <int>
```

2.2 Case-by-Variable Format

- Each **row** is one case (a passenger, a flight, a car)
- Each **column** is a variable measured on that case
- **Tibbles** are the tidyverse's version of a data frame

A dataset is **tidy** when it follows three rules (more in Tidying):

1. Each variable is a column; each column is a variable
2. Each observation is a row; each row is an observation
3. Each value is a cell; each cell is a single value

2.3 Column Types

Column types determine which operations are possible. For example, you can take the `mean()` of a number (`int`, `dbl`), but not of text (`chr`).

Type	Means	Example values
`<int>`	integer (whole numbers)	`month`, `hwy`
`<dbl>`	double (decimal numbers)	`dep_delay`, `displ`
`<chr>`	character (text)	`carrier`, `class`
`<lgl>`	logical (`TRUE` / `FALSE`)	the result of `dep_delay > 0`
`<fct>`	factor (categories with a fixed order)	study strategy in Models

- Change a type with `as.numeric()`, `as.character()`, `factor()`
- !! A number stored as text (`"10"`) looks fine but `mean()` fails on it. Check `glimpse()` first.

2.4 The dplyr Rules

Every dplyr verb follows the same rules:

1. First argument is the data frame
2. Columns are named without quotes
    - !! **Exception, pivots:** when a function *creates* new columns from text, the new names go in quotes: `pivot_longer(cols = ..., names_to = "season", values_to = "count")`. Existing columns (`cols =`) are still unquoted. See Tidying.
    - Joins also accept quoted names: `left_join(y, by = "county_id")`
3. Output is always a **new** data frame (the original is never changed unless you save over it with `<-`)

2.5 The Pipe |>

The pipe means "and then".

- Passes the result on its left into the first argument of the function on its right
- Because of rule 1, every dplyr verb can be chained

```
# "Take flights, AND THEN keep Seattle flights, AND THEN count them by carrier"
flights |>
  filter(dest == "SEA") |>
  count(carrier, sort = TRUE)
```

```
# A tibble: 5 x 2
  carrier     n
  <chr>   <int>
1 DL       1213
2 UA       1117
3 AS        714
4 B6        514
5 AA        365
```

3 Rows & Columns

3.1 Quick Reference: Row Operations

Use Case	Function	Example
Keep rows that meet a condition	`filter()`	`flights |> filter(dep_delay > 120)`
Comparison for conditional statements	`== != > >= < <=`	`filter(dest == "IAH")`
Require both conditions (and)	`&` or `,`	`filter(month == 1 & day == 1)`
Accept either condition (or)	`|`	`filter(month == 1 | month == 2)`
Match any of several values	`%in%`	`filter(month %in% c(1, 2))`
Keep values inside a range	`between()`	`mpg |> filter(between(hwy, 20, 30))`
Find rows with a missing value	`is.na()`	`filter(is.na(dep_time))`
Sort rows smallest to largest	`arrange()`	`flights |> arrange(year, month, day, dep_time)`
Sort rows from largest to smallest	`desc()`	`flights |> arrange(desc(dep_delay))`
Keep the row(s) with the largest value	`slice_max()`	`mpg |> slice_max(hwy, n = 3)`
Keep the row(s) with the smallest value	`slice_min()`	`mpg |> slice_min(hwy)`
Keep rows by position	`slice()`	`mpg |> slice(1:5)`
Keep unique rows or combinations	`distinct()`	`flights |> distinct(origin, dest)`

Use Case	Function	Example
Keep the other columns	`.keep_all = TRUE`	`distinct(origin, dest, .keep_all = TRUE)`
Count how often each value appears	`count()`	`flights |> count(origin, dest, sort = TRUE)`

- `slice_max()` vs. `arrange(desc()) |> slice(1)`: same idea, but `slice_max()` keeps **ties** (from HW1, where two counties could share the top value)

3.2 Quick Reference: Column Operations

Use Case	Function	Example
Add a column calculated from others	`mutate()`	`flights |> mutate(speed = distance / air_time * 60)`
Put the new column at the front	`.before` / `.after`	`mutate(gain = dep_delay - arr_delay, .before = 1)`
Keep only the columns used in the calculation	`.keep = "used"`	`mutate(gain = dep_delay - arr_delay, .keep = "used")`
Pick one of two values by a condition	`if_else()`	`mutate(late = if_else(arr_delay > 0, "Late", "On time"))`
Keep only some columns	`select()`	`flights |> select(year, month, day)`
Pick columns by name pattern	`starts_with()` `ends_with()` `contains()`	`select(starts_with("dep"))`
Pick columns by type	`where()`	`select(where(is.character))`
Rename a column	`rename()`	`rename(tail = tailnum)` (new name first)

3.3 Worked Example: Point Values for a Deck of Cards (`mutate()`)

From the 10/12 in-class activity

Question: In a standard 52-card deck, give every card a point value:

- Ace is worth 1
- 2 through 10 are worth their rank
- Face cards Jack, Queen and King are worth 10

Pseudocode:

1. Read in the deck
2. Make `point_value`: if the rank is Ace, use 1; otherwise copy the rank
3. Overwrite `point_value` with 10 for face cards
4. `Rank` is text (`"Ace"`, `"King"`), so convert `point_value` to a number at the end

```
deck <- read_csv("data/card_deck.csv")
```

```
Rows: 52 Columns: 4
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (3): Suit, Rank, Face
dbl (1): Card

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
deck_points <- deck |>
  mutate(
    point_value = if_else(Rank == "Ace", "1", Rank),
    point_value = if_else(Face == "Yes", "10", point_value),
    point_value = as.numeric(point_value),
    .after = Rank
  )

deck_points |> slice(c(1, 10, 13))
```

```
# A tibble: 3 x 5
   Card Suit  Rank  point_value Face
  <dbl> <chr> <chr>       <dbl> <chr>
1     1 Clubs Ace             1 No
2    10 Clubs 10             10 No
3    13 Clubs King           10 Yes
```

!! In Notes #1 this chunk started with `filter(Face == "No")`, which removed the face cards before the `Face == "Yes"` line could ever run. Filter *after* assigning values if you only want some cards:

```
# Average point value of the non-face cards
deck_points |>
  filter(Face == "No") |>
  summarize(avg_value = mean(point_value))
```

```
# A tibble: 1 x 1
  avg_value
      <dbl>
1       5.5
```

Answer: Every card now has a numeric `point_value`. The 40 non-face cards average 5.5 points.

3.4 Worked Example: Flights to Seattle with the Longest Delays

Question: Which flights from NYC to Seattle (SEA) arrived the latest?

Pseudocode: keep Seattle flights → sort by arrival delay, biggest first → show only the columns that identify the flight.

```r
flights |>
  filter(dest == "SEA") |>
  arrange(desc(arr_delay)) |>
  select(month, day, carrier, origin, dest, arr_delay)
```

```
# A tibble: 3,923 x 6
   month   day carrier origin dest  arr_delay
   <int> <int> <chr>   <chr>  <chr>     <dbl>
 1     6    24 DL      JFK    SEA         444
 2     6    27 DL      JFK    SEA         400
 3     9    12 DL      JFK    SEA         336
 4     8    11 B6      JFK    SEA         328
 5     7    10 B6      JFK    SEA         307
 6     3    18 UA      EWR    SEA         299
 7     6    27 DL      JFK    SEA         292
 8     9    12 UA      EWR    SEA         269
 9     7    22 B6      JFK    SEA         258
10     7    22 UA      EWR    SEA         255
# i 3,913 more rows
```

Answer: Out of 3,923 flights to Seattle, the latest was a Delta flight from JFK on June 24, which arrived 444 minutes (7.4 hours) late.

4 Groups & Summaries

4.1 Quick Reference

Use Case	Function	Example
Split the data into groups	`group_by()`	`mpg |> group_by(class)`
Collapse each group to one row of results	`summarize()`	`summarize(avg_hwy = mean(hwy))`
Count rows per group	`n()`	`summarize(n = n())`
Count rows per group (shortcut)	`count()` / `tally()`	`mpg |> count(class, sort = TRUE)`
Find the center	`mean()` / `median()`	`summarize(mean_hwy = mean(hwy))`
Find the spread	`sd()` / `IQR()`	`summarize(sd_hwy = sd(hwy), iqr_hwy = IQR(hwy))`
Find the extremes	`min()` / `max()`	`summarize(min_hwy = min(hwy), max_hwy = max(hwy))`
Find a percentile	`quantile()`	`quantile(hwy, 0.25)`
Ignore missing values	`na.rm = TRUE`	`mean(dep_delay, na.rm = TRUE)`
Get the first or last value	`first()` / `last()`	`summarize(first_model = first(model))`
Group by more than one variable	`group_by(a, b)`	`group_by(year, month, day)`
Remove the grouping	`ungroup()`	`... |> ungroup()`

A full group-and-summarize pipeline:

```
mpg |>
  group_by(class) |>
  summarize(
    avg_hwy = mean(hwy),
    median_hwy = median(hwy),
    n = n()
  ) |>
  arrange(desc(avg_hwy))
```

```
# A tibble: 7 x 4
  class      avg_hwy median_hwy     n
  <chr>        <dbl>      <dbl> <int>
1 compact       28.3       27      47
2 subcompact    28.1       26      35
3 midsize       27.3       27      41
4 2seater       24.8       25       5
5 minivan       22.4       23      11
6 suv           18.1       17.5    62
7 pickup        16.9       17      33
```

4.2 Counting: `count()` versus `tally()` versus `n()`

All three count rows:

- `count()` groups and counts in one step
  - Add `sort = TRUE` to sort greatest to least
- `tally()` counts rows in data that is **already grouped** with `group_by()`
- `n()` only works inside `summarize()` or `mutate()`, so it can sit next to other summaries

```
mpg |> count(drv)
```

```
# A tibble: 3 x 2
  drv       n
  <chr> <int>
1 4       103
2 f       106
3 r        25
```

```
# use group_by() before tally()
mpg |>
  group_by(drv) |>
  tally()
```

```
# A tibble: 3 x 2
  drv       n
  <chr> <int>
1 4       103
2 f       106
3 r        25
```

```
# use n() inside summarize(), alongside other summaries
mpg |>
  group_by(drv) |>
  summarize(
    n_cars = n(),
    avg_hwy = mean(hwy)
  )
```

```
# A tibble: 3 x 3
  drv   n_cars avg_hwy
  <chr>  <int>   <dbl>
1 4        103    19.2
2 f        106    28.2
3 r         25    21
```

4.3 Worked Example: Most Delayed Departures by NYC Airport

Question: Which NYC airport has the longest average departure delay?

Pseudocode: group flights by origin airport → average the departure delay in each group (skip `NA`s for cancelled flights) → count flights per group for context → sort, biggest first.

```
flights |>
  group_by(origin) |>
  summarize(
    avg_dep_delay = mean(dep_delay, na.rm = TRUE),
    n_flights = n()
  ) |>
  arrange(desc(avg_dep_delay))
```

```
# A tibble: 3 x 3
  origin avg_dep_delay n_flights
  <chr>          <dbl>     <int>
1 EWR             15.1    120835
2 JFK             12.1    111279
3 LGA             10.3    104662
```

Answer: Newark (EWR) has the longest average departure delay at 15.1 minutes, followed by JFK (12.1) and LaGuardia (10.3).

5 Tidying & Reshaping

From the 10/19 in-class activity (Day 6)

Real data rarely arrives tidy. Before reshaping, ask: **what should one row represent?**

5.1 Quick Reference

Use Case	Function	Example
Many columns → fewer, longer columns	`pivot_longer()`	`pivot_longer(cols = -id, names_to = "year", values_to = "n")`
Many rows → fewer, wider rows	`pivot_wider()`	`pivot_wider(names_from = measure, values_from = value)`
Does a text value contain a pattern?	`str_detect()`	`filter(str_detect(name, "County"))`
Pull a number out of messy text	`parse_number()`	`parse_number("45 minutes")` → `45`
Replace text that matches a pattern	`str_replace()`	`str_replace(x, "mins", "")`
Make text all lowercase	`str_to_lower()`	`str_to_lower("March")` → `"march"`
Create a small table by hand	`tribble()`	see Joins

!! In pivots, **new** column names are quoted (`names_to = "year"`), but **existing** columns are not (`cols = starts_with("F_count")`, `names_from = measure`). This is the exception to dplyr rule 2 in Data Basics.

5.2 Wide → Long: `pivot_longer()`

Use when column **names** are actually values of a variable (years, seasons, episodes).

```
# A wide table: one column per year
wide <- tribble(
  ~county,  ~`2021`, ~`2022`, ~`2023`,
  "Alder",  10.1,    10.4,    10.2,
  "Birch",  12.0,    12.3,    12.5
)

wide |>
  pivot_longer(
    cols = -county,          # every column except county (unquoted: it exists)
    names_to = "year",       # quoted: this column is being created
    values_to = "estimate"   # quoted: this column is being created
  )
```

```
# A tibble: 6 x 3
  county year  estimate
  <chr>  <chr>    <dbl>
```

```
1 Alder  2021      10.1
2 Alder  2022      10.4
3 Alder  2023      10.2
4 Birch  2021      12
5 Birch  2022      12.3
6 Birch  2023      12.5
```

Arguments:

- `cols`: which existing columns to stack
- `names_to`: name of the new column that holds the old column names
- `values_to`: name of the new column that holds the cell values

5.2.1 Splitting column names into several variables

In class, the Ted Lasso data had one row and columns like `F_count_RK_S1_e1` (Roy Kent's count, season 1, episode 1) and `F_count_total_S1_e1`. `names_pattern` uses a regular expression to pull the pieces apart:

```
tl_wide |>
  pivot_longer(
    cols = starts_with("F_count_"),
    names_to = c(".value", "Season", "Episode"),
    names_pattern = "F_count_(RK|total)_S(\\d+)_e(\\d+)"
  )
```

- Each `( )` group in the pattern fills one entry of `names_to`
  - Group 1, `(RK|total)`: "RK" or "total"
  - Group 2, `S(\\d+)`: the digits after `S`, i.e. the season
  - Group 3, `e(\\d+)`: the digits after `e`, i.e. the episode
- `".value"` is special: instead of making a column called `.value`, the matched text (`RK`, `total`) **becomes column names**, so the result has one row per episode with an `RK` column and a `total` column
- `\\d+` means "one or more digits" (regex cheat sheet: the stringr regex vignette)

5.3 Long → Wide: `pivot_wider()`

Use when one case is spread across several rows (e.g. CDC PLACES has one row per county **per measure**).

```
long <- tribble(
  ~county, ~measure,   ~value,
  "Alder", "Diabetes", 10.2,
  "Alder", "Obesity",  31.0,
  "Birch", "Diabetes", 12.5,
  "Birch", "Obesity",  33.4
)
```

```r
long |>
  pivot_wider(
    id_cols = county,        # one row per county
    names_from = measure,    # new column names come from here
    values_from = value      # cell values come from here
  )
```

```
# A tibble: 2 x 3
  county Diabetes Obesity
  <chr>     <dbl>   <dbl>
1 Alder      10.2    31
2 Birch      12.5    33.4
```

- `names_from` can take several columns, e.g. `names_from = c(MeasureId, Data_Value_Type)`, which glues them into names like `DIABETES_Crude prevalence`

5.4 Cleaning Messy Survey Answers

The raw class survey had free-text answers like `"45 minutes"`, `"1 hr"`, and `"30"` in a minutes column. Write the cleaning steps in order before coding:

Pseudocode:

1. Make everything lowercase so `"Hr"` and `"hr"` match
2. Find the answers given in hours with `str_detect()`
3. Pull out the number with `parse_number()`
4. Multiply the hour answers by 60
5. Check what's left with `count()` and decide what to do with anything unusable (set to `NA`, don't guess)

Code Implementation:

```r
survey <- tibble(routine_min = c("45 minutes", "1 hr", "30", "1.5 hours", "idk"))

# INCOMPLETE - Added to To-Do.
```

6 Joins & Relational Data

From the 10/21–10/22 in-class activity (Day 7)

A join is a decision about which records belong together, not just a line of code. Before joining, ask:

1. What does **one row** represent in each table?
2. What makes two records a **valid match**?
3. After joining, does the result still answer my question? (Check the row count and what one row now means.)

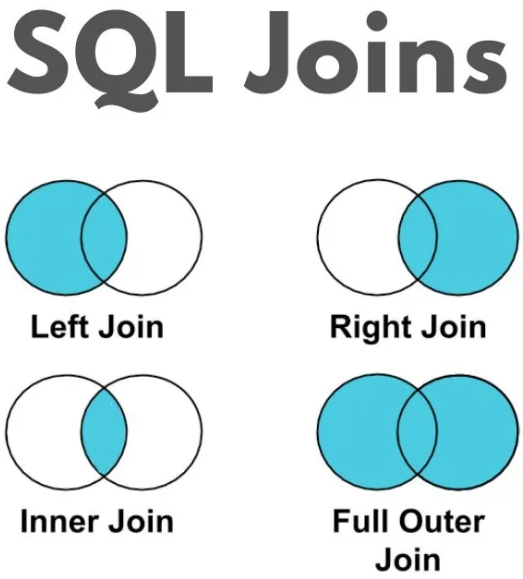


Figure 1: Source: medium.com

6.1 Quick Reference

Function	Keeps which rows?	Adds columns from `y`?	Use it to…
`left_join(x, y)`	All rows in `x`	Yes	Add lookup info to my starting table
`right_join(x, y)`	All rows in `y`	Yes	Same as a left join with `x`/`y` swapped
`inner_join(x, y)`	Only rows that match	Yes	Analyze complete matches only
`full_join(x, y)`	All rows from both	Yes	Reconcile/audit two sources
`semi_join(x, y)`	Rows in `x` with a match	No	Filter `x` to records that have a match
`anti_join(x, y)`	Rows in `x` with no match	No	Find what *didn't* match (diagnostics)
`intersect(x, y)`	Identical rows in both	No (same columns)	Rows shared by two extracts
`union(x, y)`	Distinct rows in either	No (same columns)	Stack two compatible tables, no duplicates
`setdiff(x, y)`	Rows in `x` not in `y`	No (same columns)	What was added/removed between snapshots

- Read `x |> left_join(y, by = "id")` as "**join y to x**": `x` is the starting population
- Always write `by =` (or `join_by()`) so dplyr doesn't silently match on other columns that share a name

6.2 Example Tables

```
counties <- tribble(
  ~county_id, ~county,   ~population,
  101,        "Alder",   10000,
  102,        "Birch",   20000,
  103,        "Cedar",   15000,
  104,        "Dogwood",  8000
)

health <- tribble(
  ~county_id, ~measure,   ~estimate,
  101,        "Diabetes", 10.2,
  102,        "Diabetes", 12.5,
  103,        "Diabetes",  9.8,
  105,        "Diabetes", 14.1   # 105 isn't in counties!
)

programs <- tribble(
  ~program_id, ~county_id, ~program,
  "P01",       101,        "Walking group",
  "P02",       101,        "Nutrition class",
  "P03",       103,        "Walking group",
  "P04",       104,        "Nutrition class",
  "P05",       104,        "Nutrition class"
)
```

6.3 Keys

Key	Meaning	Example here
Primary key	Uniquely identifies each row; never repeated, never `NA`	`county_id` in `counties`
Foreign key	Points to a key in another table; may repeat	`county_id` in `programs`
Duplicate key	A key value in several rows: fine for a foreign key, a data problem for a primary key	`101` appears twice in `programs`
Surrogate key	An ID made up just to identify records	`program_id` (`"P01"`)
Natural key	A meaningful attribute that already identifies the thing	A state + county FIPS code

Check that a primary key really is unique. Both of these should return **zero rows**:

```
counties |> count(county_id) |> filter(n > 1)
```

```
# A tibble: 0 x 2
# i 2 variables: county_id <dbl>, n <int>
```

```
counties |> filter(is.na(county_id))
```

```
# A tibble: 0 x 3
# i 3 variables: county_id <dbl>, county <chr>, population <dbl>
```

!! "Unique in this file" doesn't make it a good key. `county` names happen to be unique here, but county names repeat across states.

6.3.1 Relationships

Relationship	Meaning	Example
One-to-one	Each row matches at most one row on the other side	County → one population value
One-to-many	One row can match several rows	County → its programs
Many-to-many	Rows on both sides can match several rows	Students courses

A one-to-many join **adds rows**: Alder appears twice after joining `programs` because it has two programs. State what you expect so dplyr can warn you:

```
counties |>
  left_join(programs, by = "county_id", relationship = "one-to-many")
```

```
# A tibble: 6 x 5
  county_id county  population program_id program
      <dbl> <chr>        <dbl> <chr>      <chr>
1       101 Alder        10000 P01        Walking group
2       101 Alder        10000 P02        Nutrition class
3       102 Birch        20000 <NA>       <NA>
4       103 Cedar        15000 P03        Walking group
5       104 Dogwood       8000 P04        Nutrition class
6       104 Dogwood       8000 P05        Nutrition class
```

6.4 Mutating Joins (add columns)

Prediction first: joining `health` to `counties` with a left join should keep all four counties, and Dogwood (104) should get `NA`s because it has no health row.

```
counties |>
  left_join(health, by = "county_id")
```

```
# A tibble: 4 x 5
  county_id county  population measure  estimate
      <dbl> <chr>        <dbl> <chr>       <dbl>
1       101 Alder        10000 Diabetes     10.2
```

```
2       102 Birch        20000 Diabetes     12.5
3       103 Cedar        15000 Diabetes      9.8
4       104 Dogwood       8000 <NA>         NA
```

- !! Dogwood's `NA` means **no estimate**, not a diabetes rate of zero
- County 105 is dropped because the join starts from `counties`

```
# inner join: only Alder, Birch and Cedar have rows in both tables
counties |>
  inner_join(health, by = "county_id")
```

```
# A tibble: 3 x 5
  county_id county population measure  estimate
      <dbl> <chr>       <dbl> <chr>       <dbl>
1       101 Alder       10000 Diabetes     10.2
2       102 Birch       20000 Diabetes     12.5
3       103 Cedar       15000 Diabetes      9.8
```

- !! An inner join silently drops unmatched rows. An average after an inner join describes **only the matched counties**.

```
# full join: everything from both sides, NAs wherever there's no match
counties |>
  full_join(health, by = "county_id")
```

```
# A tibble: 5 x 5
  county_id county  population measure  estimate
      <dbl> <chr>        <dbl> <chr>       <dbl>
1       101 Alder        10000 Diabetes     10.2
2       102 Birch        20000 Diabetes     12.5
3       103 Cedar        15000 Diabetes      9.8
4       104 Dogwood       8000 <NA>         NA
5       105 <NA>            NA Diabetes     14.1
```

6.5 Filtering Joins (keep or drop rows, no new columns)

```
# Which counties HAVE a health estimate?
counties |> semi_join(health, by = "county_id")
```

```
# A tibble: 3 x 3
  county_id county population
      <dbl> <chr>       <dbl>
1       101 Alder       10000
2       102 Birch       20000
3       103 Cedar       15000
```

```r
# Which counties are MISSING a health estimate?
counties |> anti_join(health, by = "county_id")
```

```
# A tibble: 1 x 3
  county_id county  population
      <dbl> <chr>        <dbl>
1       104 Dogwood       8000
```

```r
# Which health rows point to a county we don't have?
health |> anti_join(counties, by = "county_id")
```

```
# A tibble: 1 x 3
  county_id measure  estimate
      <dbl> <chr>       <dbl>
1       105 Diabetes     14.1
```

The two anti joins answer different questions, so the order of `x` and `y` matters.

6.6 Set Operations (compare whole rows)

Set operations need tables with the **same columns** and compare entire rows, not a chosen key.

```r
fall <- tribble(
  ~student_id, ~course,
  1, "Data 200",
  2, "Data 200",
  3, "Data 232"
)
spring <- tribble(
  ~student_id, ~course,
  2, "Data 200",
  3, "Data 232",
  4, "Data 200"
)

intersect(fall, spring)  # identical rows in both terms
```

```
# A tibble: 2 x 2
  student_id course
       <dbl> <chr>
1          2 Data 200
2          3 Data 232
```

```r
union(fall, spring)      # every distinct row, once
```

```
# A tibble: 4 x 2
  student_id course
       <dbl> <chr>
1          1 Data 200
2          2 Data 200
3          3 Data 232
4          4 Data 200
```

```
setdiff(spring, fall)    # rows new in spring
```

```
# A tibble: 1 x 2
  student_id course
       <dbl> <chr>
1          4 Data 200
```

!! `intersect()` finds identical student-*and*-course rows. To find students enrolled in both terms *regardless of course*, compare just the key: `fall |> semi_join(spring, by = "student_id")`.

7 Plots with ggplot2

Every plot starts with the same structure:

```
ggplot(data, aes(x = ..., y = ...)) +
  geom_*()
```

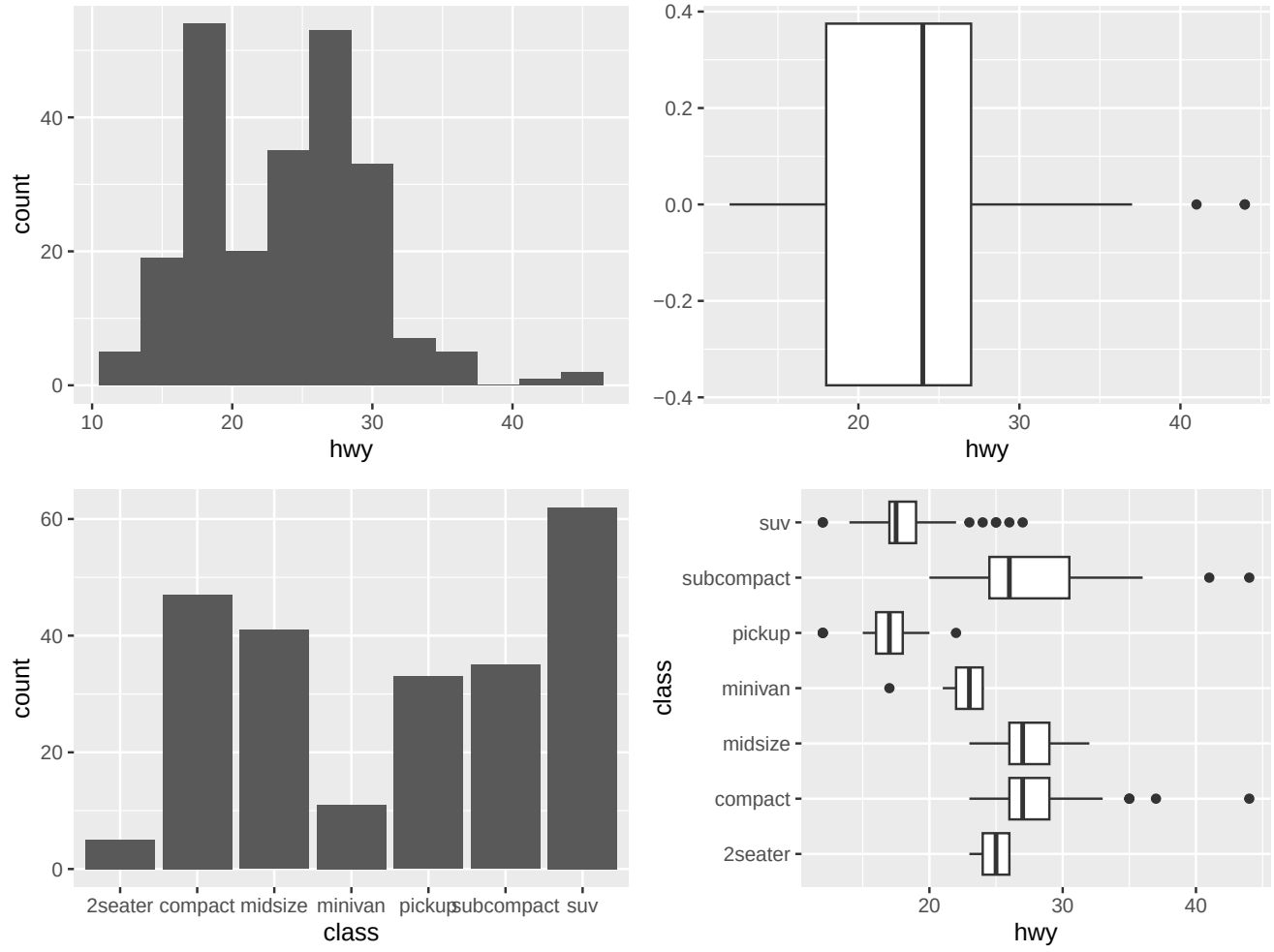
- `aes()` maps **columns** to things you can see (x, y, color, fill, size)
- **Each layer is added with a + at the end of the line**
- Pick the geom by the types of variables (more in Choosing a Plot)

7.1 Quick Reference: Geoms

Use Case	Function	Variables
Histogram: distribution of one number	`geom_histogram()`	1 numerical
Boxplot: spread and outliers	`geom_boxplot()`	1 numerical (+ 1 categorical)
Bar graph: count each category	`geom_bar()`	1 categorical
Scatterplot: relationship between two numbers	`geom_point()`	2 numerical
Jittered points: overlapping points spread out	`geom_jitter()`	2 numerical or 1 cat. + 1 num.
Count plot: point size = number of overlaps	`geom_count()`	2 discrete (e.g. ratings 1–5)
Trend line through a scatterplot	`geom_smooth()`	2 numerical
Reference line at a value	`geom_vline()` / `geom_hline()`	any
Small multiples: one panel per group	`facet_wrap(~ group)`	any + 1 categorical

7.1.1 Examples

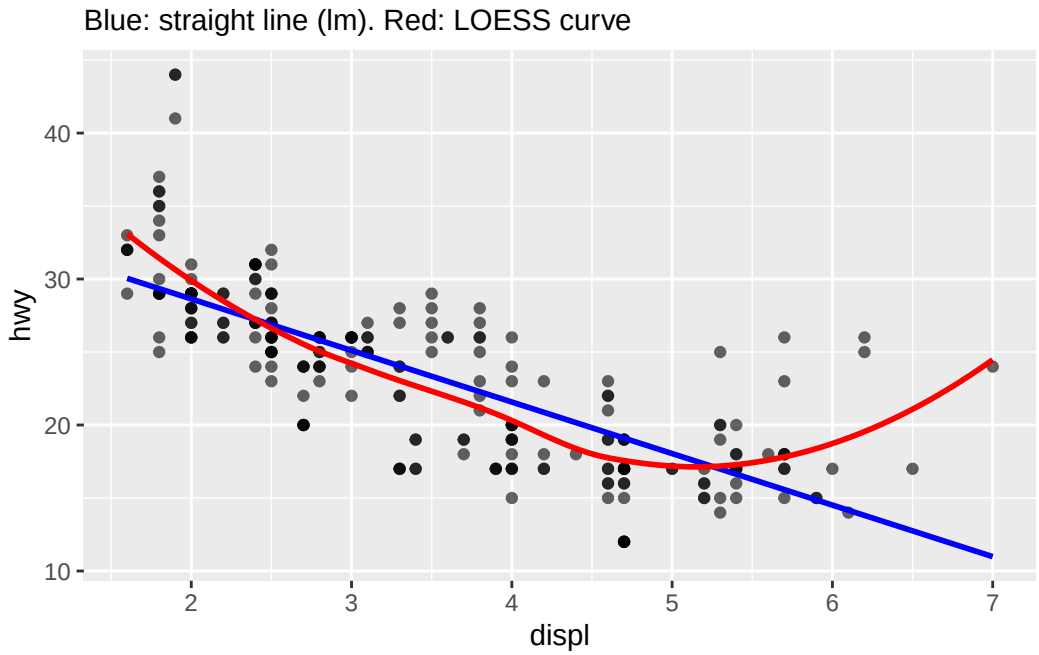
```
# Histogram: play around with binwidth
ggplot(mpg, aes(x = hwy)) +
  geom_histogram(binwidth = 3)
# Boxplot of one variable
ggplot(mpg, aes(x = hwy)) +
  geom_boxplot()
# Bar graph: counts each category for you
ggplot(mpg, aes(x = class)) +
  geom_bar()
# Boxplot comparing a number across categories
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_boxplot() +
  coord_flip() # flip so long category names don't overlap
```



7.2 Trend Lines (`geom_smooth()`)

From the Day 5 EDA activity

```r
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE, color = "blue") +
  geom_smooth(method = "loess", formula = y ~ x, se = FALSE, color = "red") +
  labs(subtitle = "Blue: straight line (lm). Red: LOESS curve")
```



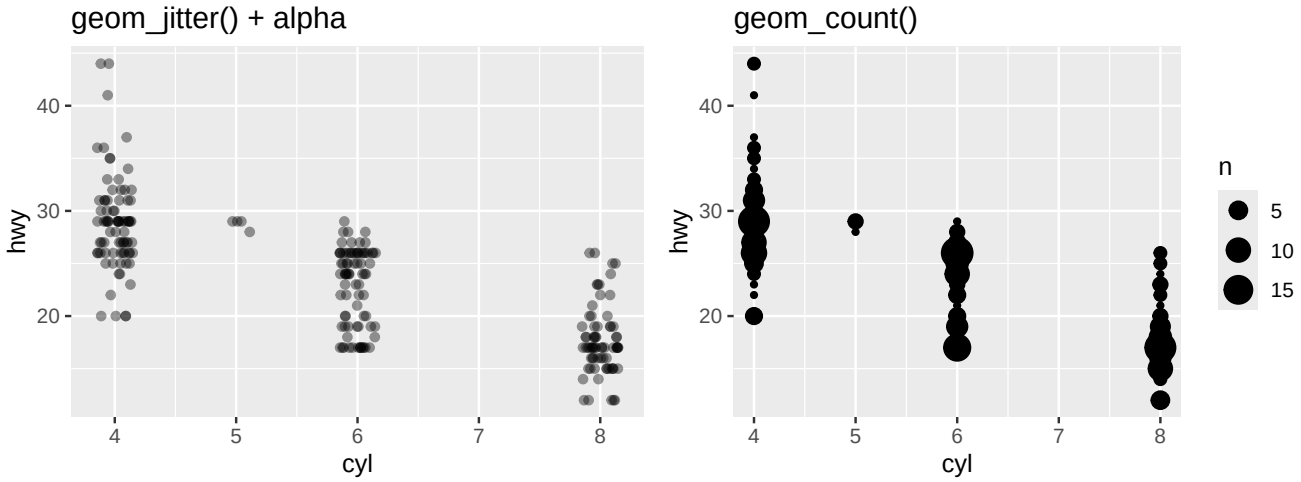
Argument	What it does
`method = "lm"`	Straight line of best fit (a linear model, see Models)
`method = "loess"`	Flexible curve that follows the local pattern (the default for small data)
`se = FALSE`	Removes the gray confidence band around the line
`formula = y ~ x`	Spells out the formula and silences the "using formula" message

- !! If the straight line and the curve disagree (like above for big engines), a single straight line is a poor summary
- `stat_smooth()` does the same thing as `geom_smooth()`

7.3 Overplotting: Jitter, Transparency, Count

From the Day 8 activity. When values are whole numbers (ratings 1–5), hundreds of points land on the same spot and the plot hides how many there are.

```r
ggplot(mpg, aes(x = cyl, y = hwy)) +
  geom_jitter(width = 0.15, height = 0, alpha = 0.4) +
  labs(title = "geom_jitter() + alpha")
ggplot(mpg, aes(x = cyl, y = hwy)) +
  geom_count() +
  labs(title = "geom_count()")
```

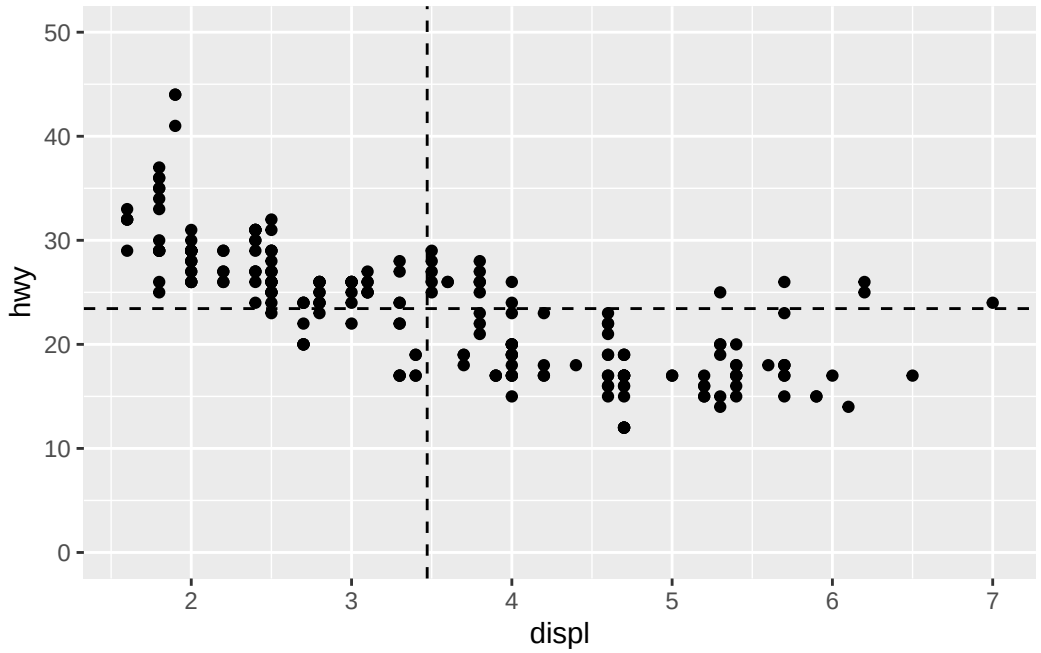


- `geom_jitter(width =, height =)` nudges each point a little at random so stacks spread out
  - !! Jitter only changes where points are **drawn**, not the data. Set `height = 0` when the y values must stay exact
- `alpha = 0.4` makes points see-through: darker spots = more points
- `geom_count()` keeps points in place and sizes them by count. Downside: people judge areas poorly

7.4 Reference Lines and Axis Limits

From HW1

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_vline(xintercept = mean(mpg$displ), linetype = "dashed") +
  geom_hline(yintercept = mean(mpg$hwy), linetype = "dashed") +
  scale_y_continuous(limits = c(0, 50))
```

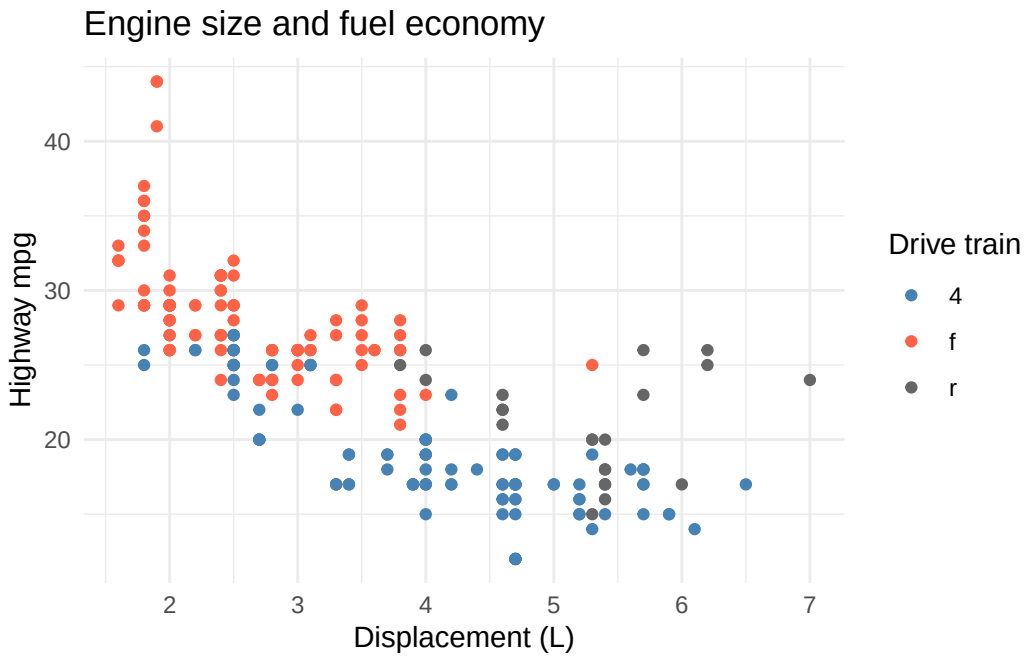


- `geom_vline(xintercept =)` draws a vertical line; `geom_hline(yintercept =)` draws a horizontal one
- `mpg$hwy` pulls one column out of a data frame as a vector (needed here because the line isn't inside `aes()`)
- `scale_y_continuous(limits = c(0, 50))` sets the axis range. Starting at 0 avoids exaggerating differences

7.5 Quick Reference: Plot Styling

Use Case	Function	Example
Add a title, subtitle and axis labels	`labs()`	see the chunk below
Rename the legend	`labs(fill = )` / `labs(color = )`	`labs(fill = "Fare Class")`
Color by a variable	`aes(color = )` / `aes(fill = )`	`aes(x = displ, y = hwy, color = class)`
Choose a color for each category (bars)	`scale_fill_manual()`	see the chunk below
Choose a color for each category (points, lines)	`scale_color_manual()`	see the chunk below
List R's color names	`colors()`	Run in the Console; hex codes work too: `"#4682B4"`
Change the overall look	`theme_*()`	`theme_minimal()`, `theme_bw()`, `theme_classic()`
Save a plot as an object	`<-`	`my_plot <- ggplot(mpg, aes(x = class)) + geom_bar()`
Export a plot as an image file	`ggsave()`	`ggsave("mpg-plot.png", plot = my_plot)`

```r
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  scale_color_manual(
    values = c("4" = "steelblue", "f" = "tomato", "r" = "gray40")
  ) +
  labs(
    title = "Engine size and fuel economy",
    x = "Displacement (L)",
    y = "Highway mpg",
    color = "Drive train"
  ) +
  theme_minimal()
```



`scale_fill_manual()` works the same way for bars (from the Titanic activity):

```r
scale_fill_manual(values = c("First" = "cornflowerblue",
                             "Second" = "mediumvioletred",
                             "Third" = "darkgoldenrod1"))
```

8 Exploratory Data Analysis

From the R4DS Ch. 10 reading and the 10/14 in-class activity (Day 5), using `mpg`

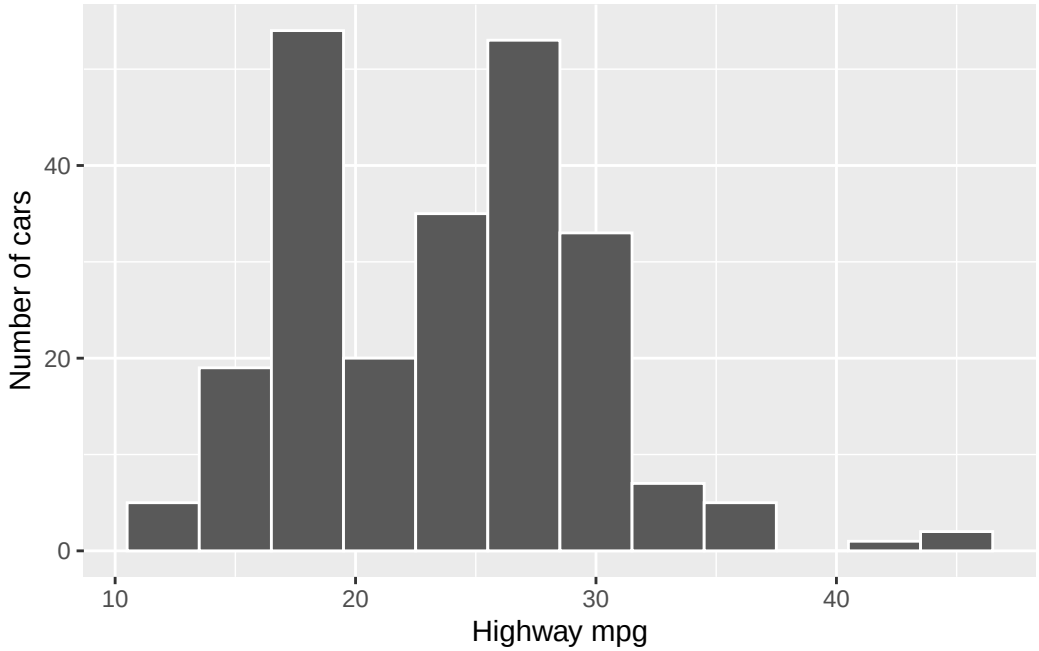
EDA means looking at the data **before** building a model. Five questions to work through:

Idea	Question	Go-to tools
Variation	How much do values differ from case to case?	`geom_histogram()`, `sd()`, `IQR()`
Typical values	What's a reasonable summary of the "middle"?	`mean()`, `median()`, `summary()`

Idea	Question	Go-to tools
Unusual values	Which cases stand out? Errors, rare cases, or informative?	`geom_boxplot()`, 1.5 × IQR rule
Covariation	How do two variables change together?	`geom_point()`, boxplots by group, `color =`
Patterns & models	Is the trend linear, curved, clustered?	`geom_smooth()`

8.1 Variation and Typical Values

```
ggplot(mpg, aes(x = hwy)) +
  geom_histogram(binwidth = 3, color = "white") +
  labs(x = "Highway mpg", y = "Number of cars")
```



When describing a distribution, cover:

- **Center:** where most values sit (here, two bumps around 17–20 and 26–27 mpg)
- **Shape:** symmetric or skewed? One peak or several? (here: two peaks, slight right skew)
- **Spread:** how far apart values are
- **Unusual values:** anything far from the rest (a couple of cars above 40 mpg)

```
mpg |>
  summarize(
    mean_hwy = mean(hwy),
    median_hwy = median(hwy),
    sd_hwy = sd(hwy),
    iqr_hwy = IQR(hwy)
  )
```

```
# A tibble: 1 x 4
  mean_hwy median_hwy sd_hwy iqr_hwy
     <dbl>      <dbl>  <dbl>   <dbl>
1     23.4         24   5.95       9
```

8.1.1 Measures of center and spread

Measure	What it is	Affected by outliers?	Use when…
Mean	Average of all values	**Yes**, pulled toward them	Data is roughly symmetric
Median	Middle value (50th percentile)	No (resistant)	Data is skewed or has outliers
Standard deviation	Typical distance of values from the mean	**Yes**	Paired with the mean
IQR	Q3 (75th pct.) − Q1 (25th pct.): the spread of the middle 50%	No (resistant)	Paired with the median

- !! Mean far from median is a clue the data is skewed: the mean gets dragged toward the long tail
- Same median, different IQR = same center, different spread

8.2 Unusual Values

A boxplot flags a point as an outlier if it's more than **1.5 × IQR** beyond the box (below Q1 or above Q3).

```
mpg |>
  mutate(
    lower_limit = quantile(hwy, 0.25) - 1.5 * IQR(hwy),
    upper_limit = quantile(hwy, 0.75) + 1.5 * IQR(hwy)
  ) |>
  filter(hwy < lower_limit | hwy > upper_limit) |>
  select(manufacturer, model, class, hwy)
```

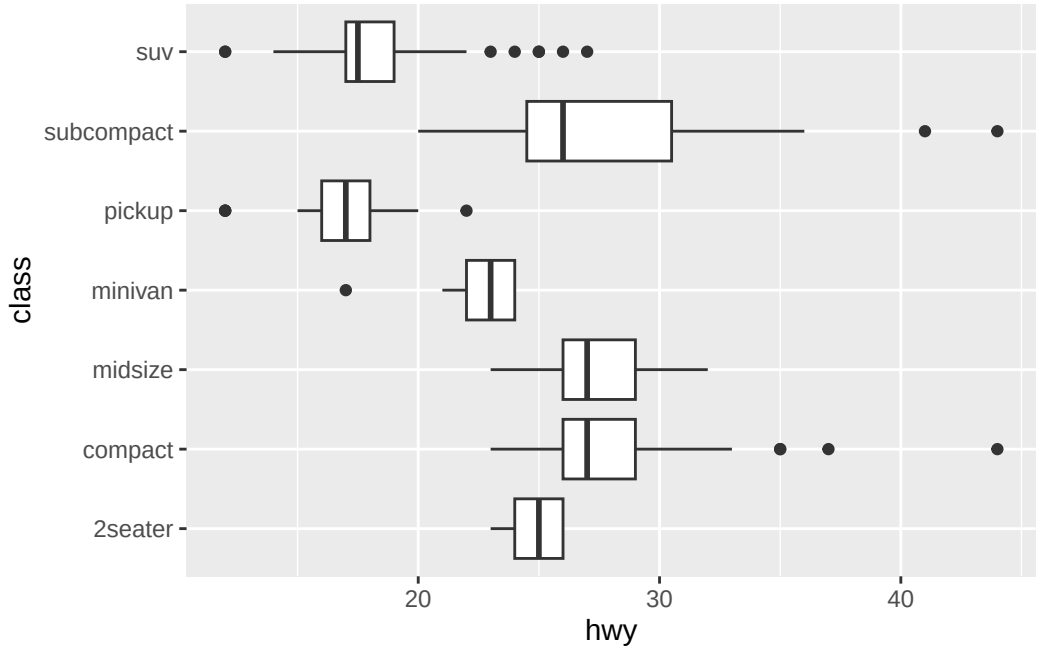
```
# A tibble: 3 x 4
  manufacturer model      class        hwy
  <chr>        <chr>      <chr>      <int>
1 volkswagen   jetta      compact       44
2 volkswagen   new beetle subcompact    44
3 volkswagen   new beetle subcompact    41
```

- Three cars are outliers, but the boxplot only shows two dots: two of them have the **same** `hwy` value, so their points overlap
- Before removing an outlier, ask: is it a data entry error, or a real (rare) case? What do the outliers have in common?
- !! "Outlier" depends on the comparison group. Split `mpg` by `class` and new outliers appear within each class.

8.3 Covariation

One numerical + one categorical: compare distributions across groups.

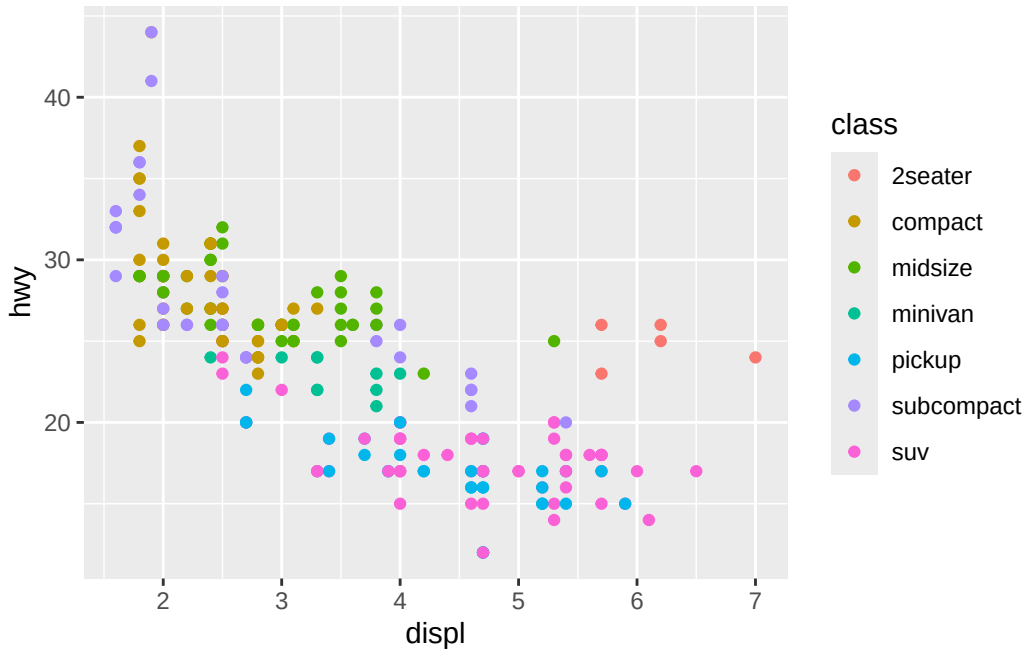
```
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_boxplot() +
  coord_flip()
```



Two numerical: scatterplot. Describe the relationship with four words:

- **Direction:** positive or negative?
- **Form:** linear or curved?
- **Strength:** weak, moderate, or strong?
- **Exceptions:** which points don't follow the pattern?

```
ggplot(mpg, aes(x = displ, y = hwy, color = class)) +
  geom_point()
```



Bigger engines get worse highway mileage (negative, slightly curved, moderately strong). Coloring by `class` explains many exceptions: the 2-seaters have big engines but better mileage than SUVs and pickups with similar engines.

8.4 Patterns and Models

Adding a trend line helps judge **form**. See Trend Lines for a straight line vs. a LOESS curve on this same plot. If the two disagree, a single straight line hides a real pattern.

9 Choosing Plots & Models

From the 10/26 in-class activity (Day 8)

A **plot** helps you see and communicate a pattern. A **model** puts a number on a relationship. You don't need a model every time you make a plot, and you should always plot before you model.

9.1 Choosing a Plot

Start with the question, then name the **outcome** (what I'm trying to understand) and the **predictor** (what might explain it), and their types.

Outcome	Predictor	First plot to try	Possible model
Numerical	Numerical	Scatterplot (+ trend line)	Linear regression
Numerical	Categorical	Jittered points or boxplots by group, means marked	ANOVA / linear model with a categorical predictor
Categorical	Categorical	Two-way table; bar chart of **proportions**	Chi-square test

Outcome	Predictor	First plot to try	Possible model
Binary (yes/no)	Numerical	Proportion "yes" across groups; jittered 0/1 points	Logistic regression
Count	Time	Counts over time	A count model (beyond this class)

- These are starting points, not rules
- !! A scatterplot is the wrong choice when both variables are categorical: every point lands on a few grid spots

9.2 Correlation

Correlation (r) measures the **direction** and **strength** of a **linear** relationship between two numerical variables, from -1 to 1.

- Positive: big x goes with big y. Negative: big x goes with small y
- Near 0: little *linear* relationship, **not** "no relationship"
- !! A U-shaped curve can have $r \approx 0$ even though y depends completely on x. Always look at the plot first.

```
cor(mpg$displ, mpg$hwy)
```

```
[1] -0.76602
```

9.3 Simple Linear Regression

The fitted line:

$$\hat{y} = b_0 + b_1 x$$

Term	Meaning
Slope b_1	Predicted change in y for a **one-unit increase** in x
Intercept b_0	Predicted y when x = 0 (often an extrapolation, so not meaningful)
Residual	Observed y − predicted y: the vertical gap between a point and the line

9.3.1 Worked Example: Study Hours and Exam Scores

Question: How much does the predicted exam score go up per extra hour of studying?

Pseudocode: plot hours vs. score → fit a line with `lm()` → read the slope with `coef()` → interpret it with units.

```r
model_data <- tibble(
  study_hours = c(1, 2, 2, 3, 4, 4, 5, 6, 7, 8, 8, 9),
  score       = c(52, 55, 61, 60, 64, 70, 68, 74, 77, 79, 85, 87)
)

fit <- lm(score ~ study_hours, data = model_data) # outcome ~ predictor
coef(fit)
```

```
(Intercept) study_hours
  48.939810    4.147835
```

- `lm()` makes the model but prints nothing; save it with `<-` and use `coef()` to see the numbers
- !! Formula order is `outcome ~ predictor` (y ~ x)

```r
# predicted values and residuals for each student
model_data |>
  mutate(predicted = fitted(fit), residual = resid(fit)) |>
  head(4)
```

```
# A tibble: 4 x 4
  study_hours score predicted residual
        <dbl> <dbl>     <dbl>    <dbl>
1           1    52      53.1    -1.09
2           2    55      57.2    -2.24
3           2    61      57.2     3.76
4           3    60      61.4    -1.38
```

Answer: Each extra hour studied goes with about 4.1 more points, on average. The intercept (48.9) is the predicted score at 0 hours, which is outside the data (the minimum is 1 hour). A positive residual means the student scored above the line.

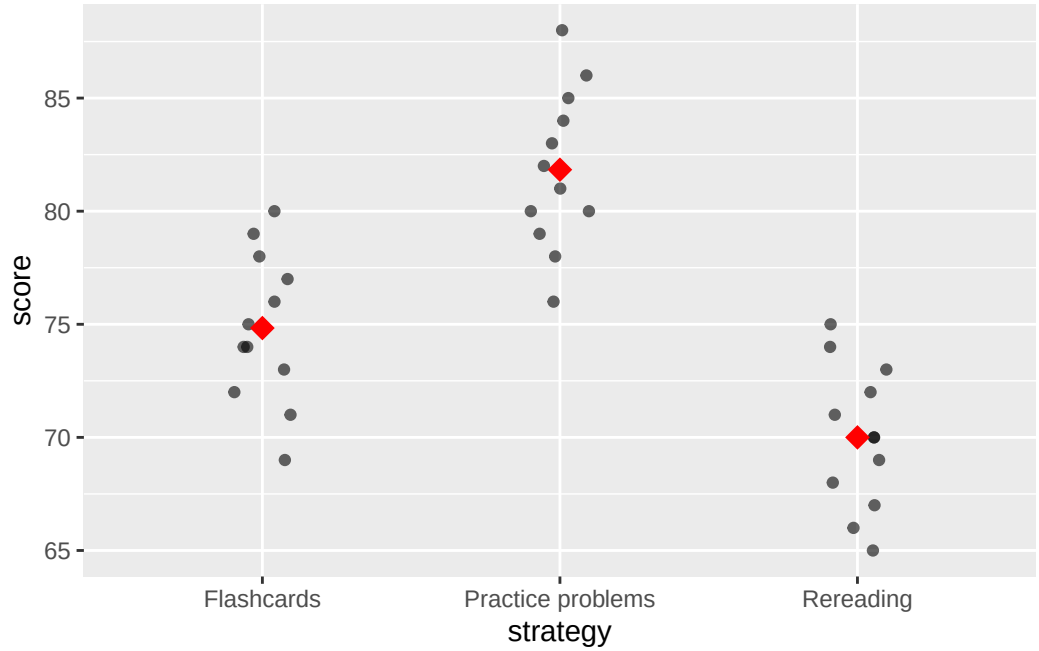
!! This is **association, not causation**: these are observational data, and motivation or prior knowledge could drive both study time and scores.

9.4 Categorical Predictor = Comparing Group Means

The same `lm()` works with a categorical predictor. It then compares **group means**.

```r
scores <- tibble(
  strategy = rep(c("Flashcards", "Practice problems", "Rereading"), each = 12),
  score = c(72, 75, 78, 69, 74, 80, 76, 73, 77, 71, 79, 74,
            78, 82, 85, 80, 76, 88, 83, 81, 79, 86, 84, 80,
            65, 72, 70, 68, 75, 71, 69, 73, 67, 74, 70, 66)
)

ggplot(scores, aes(x = strategy, y = score)) +
  geom_jitter(width = 0.1, height = 0, alpha = 0.6) +
  stat_summary(fun = mean, geom = "point", shape = 18, size = 4, color = "red")
```




```
group_fit <- lm(score ~ strategy, data = scores)
coef(group_fit)
```

```
          (Intercept) strategyPractice problems         strategyRereading
            74.833333                   7.000000                  -4.833333
```

- R picks a **reference group** (first alphabetically: Flashcards)
- **Intercept** = mean of the reference group
- **Other coefficients** = difference between that group's mean and the reference group's mean
  - e.g. `strategyPractice problems` = 7: practice problems averaged that many points more than flashcards
- Changing the reference group (`relevel()`) changes how the coefficients are written, not the group means

9.4.1 Where ANOVA fits

A **one-way ANOVA** asks: do the mean outcomes differ across two or more groups? It compares the variation **between** group means to the variation **within** groups.

```
anova(group_fit)
```

```
Analysis of Variance Table

Response: score
           Df Sum Sq Mean Sq F value    Pr(>F)
strategy    2 849.56  424.78  38.161 2.611e-09 ***
Residuals 33 367.33   11.13
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- A tiny p-value means the group means differ by more than chance would explain
- With exactly two groups, ANOVA, a two-sample t-test, and `lm()` give the same test
- !! Lead with the **plot**, not the table: means alone hide spread, overlap and outliers

10 Study Design & Data Ethics

From the 10/28 in-class activity (Day 9)

The design of a study decides which conclusions the data can support, before any code is written.

10.1 Study Designs

Design	What happens	Can show causation?	Watch out for
Randomized experiment	Researcher randomly **assigns** a treatment	Yes, within the sample	Volunteers may not represent everyone; short time frames
Observational, cross-sectional	Measure people once, at one point in time	No (association)	Confounders; self-report errors
Observational, longitudinal	Measure the same people repeatedly over time	No, but shows change within people	**Attrition** (people drop out, often not at random)
Observational, retrospective	Use records that already exist (sales, gym entries)	No	Records weren't made to answer this question

- **Confounding variable:** a third variable related to both the predictor and the outcome (e.g. screen time could affect both sleep and GPA)
- **Before-and-after** comparisons with no comparison group can't separate the change from everything else that changed (season, prices, marketing)

!! **Random assignment random sampling**

- Random **assignment** (who gets which treatment) → supports **causal** conclusions
- Random **sampling** (who is in the study) → supports **generalizing** to the population
- An experiment on volunteers at one company shows cause *for those volunteers*, but not necessarily for everyone

10.2 Ways to Collect Data

Method	Good for	Limitation
Survey / self-report	Internal experiences (stress, mood, opinions)	Recall error, social desirability, nonresponse
Interviews / focus groups	How people *describe* experiences, in their words	Small samples; stigma can cause underreporting
Wearables / tracking	Objective, repeated measures (sleep, steps)	Device errors; only reaches people who own one

Method	Good for	Limitation
Naturally generated records	Large volumes of real behavior, cheap	Not designed for research; missing context
Web scraping / APIs	Public data at scale (prices, posts, sports stats)	Site terms, changing page layouts; users aren't representative

A data collection plan should name: target population, data source, collection method, **measurements with units/scales and timing**, and a limitation.

10.3 Consent & Privacy

- **Consent** = a meaningful chance to understand what participation involves and to say no. One "Accept all" click isn't informed consent for every later use.
- **Purpose shift:** using data for a different purpose than people expected (site analytics → targeted housing ads) raises the privacy risk
- **"Anonymous" isn't unidentifiable:** browsing patterns + IP address + device details, or a wearable device's daily routine, can act as a fingerprint

10.4 Writing Unbiased Survey Questions

Problem	Example	Fix
Leading / loaded	"Don't you agree the *overpriced* dining…"	Neutral wording, no opinion words
Double-barreled	Asks about affordability *and* eating well in one question	One concept per question
Vague	"Do you get *enough* sleep?"	Ask for a number, with units
No time frame	"How often do you…"	"During the past 7 days…"
Missing options	Only "car" or "bus" for commuting	Exhaustive options, "select all", "Other"
Overlapping bins	1–2 hrs, 2–3 hrs	Non-overlapping: 1–2, 3–4
Measures the wrong thing	"How much does social media *harm* you?" when the goal is time spent	Measure the predictor and outcome separately

11 Common Pitfalls & Troubleshooting

11.1 = instead of ==

```
# !! one = means "assign" or "name an argument", not "is equal to"
flights |>
  filter(month = 1)
```

```
Error in `filter()`:
! We detected a named input.
i This usually means that you've used `=` instead of `==`.
i Did you mean `month == 1`?
```

dplyr catches this and suggests the fix:

```
flights |>
  filter(month == 1) |>
  count(month)
```

```
# A tibble: 1 x 2
  month     n
  <int> <int>
1     1 27004
```

11.2 Forgetting `na.rm = TRUE`

```
# !! any missing value makes the whole result NA
flights |>
  summarize(avg_delay = mean(arr_delay))
```

```
# A tibble: 1 x 1
  avg_delay
      <dbl>
1        NA
```

```
# Fix: ignore missing values
flights |>
  summarize(avg_delay = mean(arr_delay, na.rm = TRUE))
```

```
# A tibble: 1 x 1
  avg_delay
      <dbl>
1      6.90
```

11.3 Using the `&` and `|` Operators

```
# Keep only flights in January OR February
# !! PRODUCES A BUGGED OUTPUT
flights |>
  filter(month == 1 | 2) |>
  count(month)
```

```
# A tibble: 12 x 2
   month     n
   <int> <int>
 1     1 27004
 2     2 24951
 3     3 28834
 4     4 28330
 5     5 28796
 6     6 28243
 7     7 29425
 8     8 29327
 9     9 27574
10    10 28889
11    11 27268
12    12 28135
```

Error: R treats nonzero numbers in conditional statements as `TRUE`, so it reads this as "keep rows where `month` is 1, **or** `TRUE`", which keeps every row.

```
# Fix 1
flights |>
  filter(month == 1 | month == 2) |>
  count(month)
```

```
# A tibble: 2 x 2
  month     n
  <int> <int>
1     1 27004
2     2 24951
```

```
# Fix 2
flights |>
  filter(month %in% c(1, 2)) |>
  count(month)
```

```
# A tibble: 2 x 2
  month     n
  <int> <int>
1     1 27004
2     2 24951
```

11.4 Forgetting to Save with <-

```
flights |>
  filter(month == 1) |>
  nrow()
```

```
[1] 27004
```

```
nrow(flights) # unchanged!
```

```
[1] 336776
```

dplyr verbs never change the original data. To keep the result, save it: `jan_flights <- flights |> filter(month == 1)`.

11.5 Filtering Before a Step That Needs the Rows

!! From my own Notes #1: filtering to non-face cards *before* assigning face-card values meant the face-card rule never ran. Think through the order of steps (write pseudocode) before chaining. See the card deck example.

11.6 Quoting Column Names in Pivots

```
wide <- tibble(id = 1:2, `2022` = c(5, 6), `2023` = c(7, 8))

# !! names_to without quotes: R looks for an object called year (and finds lubridate's year() fun
wide |>
  pivot_longer(cols = -id, names_to = year, values_to = "n")
```

```
Error in `pivot_longer()`:
! `names_to` must be a character vector or `NULL`, not a function.
```

```
# Fix: new column names are quoted
wide |>
  pivot_longer(cols = -id, names_to = "year", values_to = "n")
```

```
# A tibble: 4 x 3
     id year      n
  <int> <chr> <dbl>
1     1 2022      5
2     1 2023      7
3     2 2022      6
4     2 2023      8
```

11.7 Joins That Add Rows

!! A one-to-many join repeats rows on the "one" side. A total or average taken after the join can **double count**:

```r
counties <- tibble(county_id = c(101, 103), population = c(10000, 15000))
programs <- tibble(county_id = c(101, 101, 103), program = c("A", "B", "C"))

# Real total population is 25,000, but Alder is counted twice
counties |>
  left_join(programs, by = "county_id") |>
  summarize(total_pop = sum(population))
```

```
# A tibble: 1 x 1
  total_pop
      <dbl>
1     35000
```

Fix: summarize the "many" table first (e.g. `count(county_id)`), *then* join. Or add `relationship = "one-to-one"` to make dplyr error when there are duplicates.

11.8 `NA` Is Not Zero

After a `left_join()`, unmatched rows get `NA`. That means **unknown**, not 0. Don't replace them with 0 unless you know the true value is zero.

12 Reflection & To-do

12.1 Notes #2 Reflection

1. **What topics or concepts have been the most trouble since Notes #1?**

   I think one of the more notable topics introduced between Notes #1 and now was the section on joins. I can definitely see how incredibly useful they are when you're working with multiple data sets, which is why it's taken me more time to wrap my head around this concept. I think I did a pretty good job with my notes on joins, but I do want to start using it more on projects or activities to help me solidify my understanding.

   Day 6's class activity on cleaning up data was also really informative. In a perfect world, data comes out squeaky clean and we can jump straight to analysis, right? I actually found myself enjoying the puzzle solving nature of how to tidy up a data set. Although we didn't get around to doing a code implementation, it was still pretty fun planning out a procedure in pseudocode.

2. **Did a mistake in HW1 or class show up again later? How was it addressed?**

   I wouldn't call this a "mistake," but I still tend to forget how useful some functions and operators are in R. For example:

   - The combination `c()` function to store multiple values as a vector.
   - The `%in%` to search in a vector, as opposed to writing multiple conditionals.
   - Storing stuff in variables. It's a bit ironic since you do this a lot in other programming languages, but I haven't made it in a habit in R (besides storing new datasets, of course).

These haven't been addressed yet, but they're going into my to-do list for the next notes update.

3. **What has helped most since Notes #1?**

   …This site (lol). No, seriously, finally setting aside time to figure out how to make a site built on Quarto documents has been immensely helpful for my learning in this class and outside with my own personal projects! Splitting my master document into multiple documents was a chore, but I really like the current workflow I have for adding new content going forward. The site looks really clean — it reminds me of the R for Data Science layout with the added benefit that these are still my personalized notes, so I know how to navigate quickly!

4. **Feedback for Dr. Lisinker**

   As we approach the halfway mark this quarter, I can feel things picking up and I am pretty excited about finally getting to lay out the foundations for my data story project. I do not have any notable comments or concerns I want to bring to your attention at this time. Thank you for prefixing the activities with the class sections! I also liked the new changes you made to the Canvas modules page.

   I'll be honest, some readings (like anything from the IMS textbook) are pretty dense, but I do like the current format we have with the activities. For example, Day 9's introduction to study designs was helpful. Reading the textbook and jotting down definitions is one thing, but identifying each designs' strengths and limitations under real-life scenarios really helped solidify my understanding.

12.2 Notes #1 Reflection (archived)

1. **What is something you find yourself struggling with the most often in this class? How did you incorporate that into your cheatsheet?**

   I can see myself struggling to keep track of all the syntax, even with using Quarto. The tables featured in this assignment were copied from my own personal notes I've been maintaining since week 1. A weakness of Quarto documents is that it will always be structured as an actual document that reads from top to bottom. This can be (and is already) quite lengthy unless I split my notes up into separate Quarto documents. Alternatively, I am looking into saving my notes with the Quarto sites feature and treating each document as a separate section / web page.

2. **What is something in the class that is helping you learn?**

   Quarto! I find the ability to write markdown-styled notes along with runnable code chunks extremely useful. I used to use Notion and Obsidian for my notes, but having to flip between a window for note-taking and an IDE interrupted my learning flow.

3. **What is something that I could do to help your learning in this class?**

   I already like the current flow of the pre-class readings and working on an in-class activity each session. The only minor feedback I have would be a request for you to prefix the activity .zips with the class section (ex. `200_Day5.zip`, `232_Day6.zip`). Since I'm in both of our BDATA sections, my activities get marked as a duplicate when I go to download them.

12.3 To-do List

12.3.1 Notes #2 - October 29, 2026

- ☐ Start syncing in-class activities to the GitHub repository for archiving and "deeper dives."
- ☐ Complete the code implementation under Cleaning Messy Survey Answers
- ☐ Write a checklist for evaluating a join (*did the join actually do what it was supposed to do?*)
- ☐ Notes on `%in%` operator
- ☐ Notes on using the `c()` function
- ☐ Storing values into variables: how and when its useful in R

12.3.2 Done since Notes #1

- ☒ Replace longer code in table cells with real code chunks (Plots)
- ☒ Table of contents: the sidebar on the website, and `toc: true` in the PDF
- ☒ Chunk options `error`, `message`, `label`, `warning`, `echo`, `include` (Workflow)
- ☒ Trend lines with `geom_smooth()` (Plots)
- ☒ EDA reading notes: center, spread, outliers, variation, covariation (EDA)
- ☒ Pseudocode before worked examples
- ☒ Look up the `<-` vs `=` convention (Workflow)

13 Feedback Log

Addressing feedback and how they were incorporated in each Notes iteration.

13.1 Notes #1 - October 20, 2026

#	Feedback	What I changed	Where
1	Code in the Plots table ran into the next column in the PDF	I kept table examples short, moved longer code into chunks under the table, and checked the PDF	Plots
2	Most people use `<-` over `=` so assignment stays separate from `==`	I use `<-` for assignment now, and `=` only for function arguments	Workflow
3	Columns go unquoted, except in functions like the pivots	I added the pivot exception to rule 2, plus a pitfall for forgetting the quotes	Data Basics, Tidying
4	`echo: false` runs the code but hides it in the render	I added it to my Chunk Options table with an example	Workflow
5	Turning off warnings is useful too	I added `warning: false`, along with `message`, `error`, `include` and `label`	Workflow
6	Try a Quarto site on GitHub Pages, or a clickable table of contents	I moved my notes to a Quarto website on GitHub Pages, with one page per topic	This site! :)

#	Feedback	What I changed	Where
7	A table of contents in the PDF	I added one to the PDF version, linked from the course home page	Course home